LumaRestore AI

Hackathon Technical Solution Report

Adaptive low light image restoration with classical vision and a pretrained transformer

Field	Details
Problem ID	[Enter official problem ID]
Problem title	[Enter exact official challenge title]
Team name	[Enter registered team name]
Team members	[Enter full names and roll numbers]
GitHub repository	https://github.com/Tanish3115/Robust-Low-Light-Image-Restoration-and-Enhancement
Current base commit	8a401715b1ec0bb292752aa44d4970fe4943ecf5
Demo	Local app: http://127.0.0.1:5173 [Add public demo or video link]
Report date	7 November 2026

1 Problem Understanding and Solution Strategy

Core technical challenge

Brightening a dark photograph also makes noise, compression and colour errors more visible. A single exposure adjustment is especially unreliable when the frame contains both deep shadows and bright windows or light sources. LumaRestore addresses uneven illumination, noise amplification, colour shifts and loss of fine structure while keeping the restoration process inspectable.

The application runs locally on CPU and can use CUDA when compatible hardware is available. Uploads are bounded to 20 MB and 40 million input pixels by default, with a 2048-pixel processing long edge. These are resource controls rather than a real-time performance guarantee. The measured CPU deep inference time is about 2.25 seconds per 600 by 400 image, excluding browser and evaluation overhead.

Proposed methodology

We first measure exposure, noise, texture and illumination variation, then generate classical and neural restoration candidates independently. A spatial controller compares local exposure, colour, edge and clipping confidence to blend the candidates or retain a stronger candidate. This gives users a fast classical option, a pretrained deep option and an explainable hybrid; the evaluation shows that deep-only restoration currently gives the best average reference quality on LOL-v1.

2 Technical Architecture and Implementation

The React and TypeScript client sends images to a FastAPI service. The backend decodes and validates the image, converts it to float RGB, runs the requested method and stores outputs, intermediate maps and measured reports. References enter the evaluation stage only; they do not guide restoration. [R1]

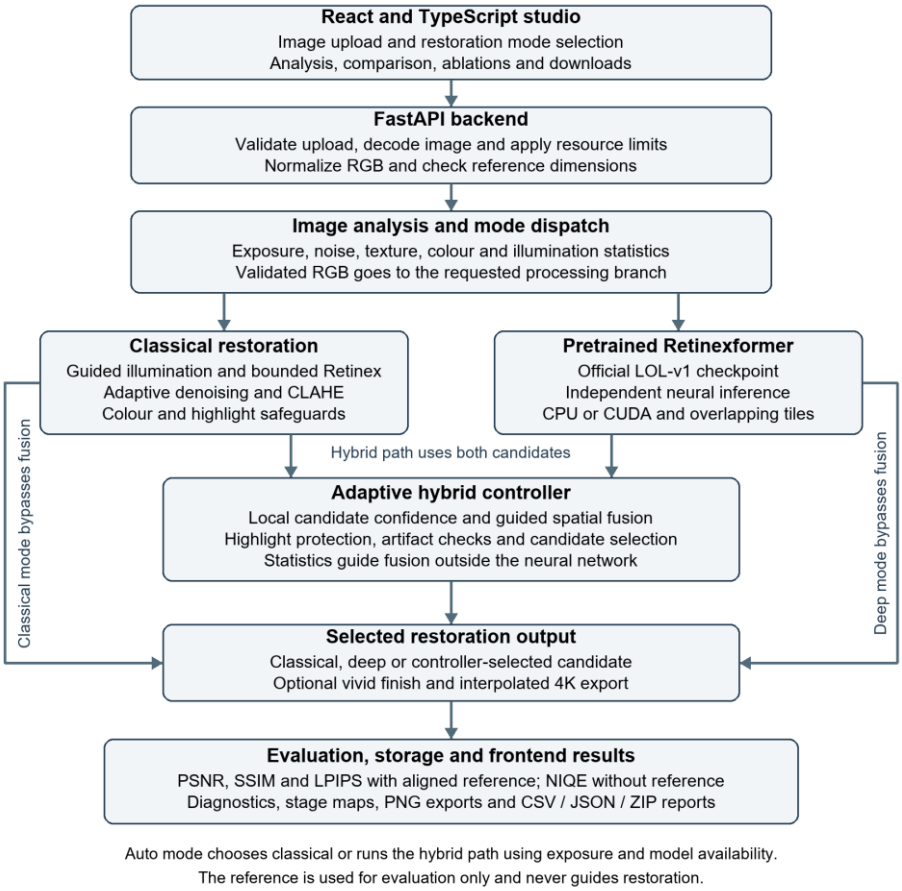


Figure 1 LumaRestore architecture and restoration data flow

Classical and deep modes return their own restoration candidates directly. Hybrid mode computes both candidates before fusion and selection; automatic mode chooses a path using exposure severity and model availability. The diagram summarizes backend/app/service.py and the branch modules in [R1].

2 1 Classical Algorithmic Baseline

The classical branch starts with a maximum-RGB illumination estimate. A luminance-guided filter smooths the estimate while retaining boundaries. The branch then applies bounded Retinex gain, bilateral denoising, luminance-only CLAHE, restrained white balance and saturation correction, followed by highlight preservation and artifact safeguards.

Exposure severity is computed as s = clip((0.43 - mean luminance) / 0.43, 0, 1). At strength k, gamma is clip(1 - 0.64sk, 0.28, 1) and maximum gain is 1 + 4.5sk. The refined illumination has a floor of 0.025 + min(2n, 0.06), where n is a flat-region Haar noise estimate. Raising this floor for noisier inputs limits unstable amplification.

Bilateral mixing increases with noise and darkness, while CLAHE mixing decreases as noise rises. The CLAHE grid adapts to image dimensions, and a support mask prevents the contrast stage from inventing illumination in completely black regions. Original highlights receive progressively stronger protection above luminance 0.65. These rules are implemented in backend/src/classical/pipeline.py; they are deterministic and do not use reference images.

2 2 Deep Learning Pipeline

Model architecture The neural branch uses the official pretrained Retinexformer LOL-v1 model in PyTorch. It combines an illumination estimator with an illumination-guided transformer restoration module. The project reuses the published architecture and checkpoint; it does not introduce a new neural network. See backend/src/deep/inference.py and backend/src/deep/vendor/retinexformer_arch.py. [R1, R2]

Illumination estimator The three-channel RGB input is concatenated with its channel-wise mean to form a four-channel representation. A 1 by 1 convolution projects this into 40 channels, followed by a grouped 5 by 5 convolution. A final 1 by 1 convolution predicts a three-channel illumination map. The adjusted input is input multiplied by the illumination map plus input. The estimator also supplies illumination features to guide restoration.

Restoration network A 3 by 3 input convolution feeds a two-level encoder with feature widths of 40, 80 and 160 channels. Illumination-guided attention blocks operate at the encoder scales and bottleneck. The decoder restores resolution using transposed convolutions, encoder skip connections and further attention blocks. A final 3 by 3 convolution predicts an RGB residual that is added to the illumination-adjusted input.

Neural network specifications

Specification	Project configuration
Framework and model	PyTorch; single-stage Retinexformer
Input and output	Three-channel RGB; inference input and clipped output in [0, 1]
Restoration stages	1
Feature widths	40, 80 and 160 channels across scales
Encoder and decoder	2 downsampling levels; 2 matching upsampling levels
Attention block configuration	[1, 2, 2] for encoder scales and bottleneck; mirrored decoder
Total block pairs	8 attention and feed-forward pairs across the full restoration module
Attention heads	1, 2 and 4 at the 40, 80 and 160 channel scales
Dimension per attention head	40
Feed-forward module	Expansion factor 4; GELU; depthwise 3 by 3 convolution
Downsampling and upsampling	4 by 4 stride-2 convolutions; 2 by 2 stride-2 transposed convolutions
Reconstruction connections	Encoder skip connections and residual RGB output
Total parameters	1,605,701; counted from the instantiated project model
FP32 parameter storage	Approximately 6.13 MiB; excludes activations and runtime overhead
Production checkpoint	Official pretrained LOL-v1 weights

The parameter count and storage estimate describe this instantiated configuration. They do not measure inference memory or include checkpoint metadata.

Deep Learning Inference and Training

Inference implementation The engine validates checkpoint weights strictly, caches one loaded model and runs in inference mode. Inputs are padded to multiples of eight. Larger images use overlapping tiles with weighted reconstruction to reduce boundary artifacts. The benchmark tile limit was 1024 pixels with 32-pixel overlap; all 600 by 400 test images fitted within one tile. CPU execution is supported, with CUDA when compatible hardware is available and recovery to CPU for supported device failures. Deep-only mode processes original RGB independently of classical enhancement and fusion.

Dataset protocol The preparation script splits the official 485-pair LOL-v1 training partition into 437 training and 48 development pairs with seed 42. The 15 official test pairs remain separate. Development images may have appeared in the official checkpoint's upstream training corpus, so they do not establish unseen pretraining generalization. Test references enter evaluation only and never guide restoration. [R3, R4]

Loss functions and training An exploratory supervised fine-tuning run used one epoch on CPU, 64 by 64 patches, batch size 2 and Adam at learning rate 0.0003. The adaptation loss combines mean Charbonnier reconstruction error with horizontal and vertical gradient L1 errors: loss = mean(sqrt((prediction - reference) squared + 0.000001)) + 0.05 times (horizontal gradient L1 + vertical gradient L1). This is the local adaptation objective, not a reproduction of the original model's training recipe. The script includes gradient clipping, a cosine learning-rate scheduler, development validation and checkpoint saving. [R5]

Measured adaptation outcome Recorded training and development losses were 0.09063 and 0.10235. On the official test set, adaptation reduced PSNR from 25.1553 to 21.5054 dB and SSIM from 0.8441 to 0.8132. The adapted checkpoint was not deployed; the application retains the original pretrained weights. NIQE improved slightly despite worse reference metrics, illustrating why NIQE alone is insufficient for model promotion. The test set was excluded from fine-tuning updates.

2 3 Hybrid Integration and Adaptive Mechanism

Classical maps and image statistics guide the external controller; they are not injected into Retinexformer or used to retrain its attention layers. Both candidates receive local confidence scores based on exposure suitability, agreement with input edges, flat-region residuals, chromaticity changes, newly clipped pixels and texture loss. These scores are uncalibrated heuristics, not probabilities of correctness.

The neural mixing weight is a sigmoid of the confidence difference plus darkness and noise priors. A guided filter smooths this weight against the original luminance. Before safeguards, the fused image is w times the deep candidate plus (1 - w) times the classical candidate. The full hybrid protects bright input regions, redirects new clipping toward a safer candidate and applies bounded artifact corrections.

The controller can reject fusion. For mildly underexposed inputs it may prefer a similarly confident classical candidate; otherwise, a candidate exceeding fused confidence by more than 0.025 can replace the fusion. The frontend exposes the selected method, explanation and contribution maps. Artifact maps identify heuristic risk and should not be interpreted as ground-truth defect labels. See backend/src/hybrid/controller.py.

Frontend and processing outputs

Six views cover restoration, image analysis, pipeline stages, independent model comparison, performance analytics and regional quality diagnostics. Analytics supports single-image ablations, batch jobs, completed benchmark reports and exports. The optional vivid finish and interpolated 4K export are presentation features; the reported benchmark and ablation runs do not apply them.

3 Experimental Benchmarks and Evaluation

3 1 Quantitative Benchmarks

We report means over the official 15-image LOL-v1 test set, with 15 valid measurements for each metric and method. Evaluation uses aligned full RGB images in [0, 1], without border cropping, reference exposure adjustment or test-time optimization. All images remain at 600 by 400 pixels. The saved run was generated on 6 November 2026. [R6]

Approach	PSNR dB higher	SSIM higher	LPIPS lower	NIQE lower	Latency ms
Classical baseline	13.5286	0.5166	0.3678	7.8276	108.2
Deep only	25.1553	0.8441	0.1314	3.4792	2247.3
Adaptive hybrid	19.1588	0.7847	0.1799	4.3247	2565.8

Table 1 Official test set averages from outputs/LOL-v1-final-benchmark/report.json

Deep-only restoration leads all four quality metrics in this experiment, reaching 25.1553 dB PSNR. The hybrid improves substantially over the classical baseline but trails deep-only by 5.9965 dB PSNR and 0.0594 SSIM. Classical processing is the fastest option at 108.2 ms; running both candidates and the controller raises hybrid latency to 2565.8 ms.

Metric interpretation and timing scope

PSNR and SSIM compare the restored image with the aligned normal-light reference; higher values indicate closer agreement. LPIPS is also reference-based and lower is better. NIQE measures departures from a natural-image statistical model without a reference; a lower score alone does not prove better restoration. Genuine LPIPS and NIQE models were enabled for these runs, with no substituted scores.

Latency is the restoration time recorded by each pipeline. Hybrid timing sums classical, neural and controller processing. It excludes upload, image decoding, report generation, perceptual-metric evaluation and browser rendering, and is not an end-to-end request benchmark. A first neural call can include initialization overhead. These are single-run CPU means, not repeated timing estimates with confidence intervals.

Measured trade-off

Average output clipping, defined as the fraction of pixels with any channel at least 254/255, was 0.4594% for deep-only and 0.0069% for hybrid. This shows a clipping trade-off, but it does not cancel the hybrid's lower reference fidelity or establish that all preserved highlights are perceptually correct. The confidence rules can favour conservative outputs that remain too dark relative to the reference.

Runtime and reproducibility

The recorded environment is Windows with Python 3.11.15, Node.js 22.14.0, PyTorch 2.7.0+cpu and torchvision 0.22.0+cpu. The benchmark sets CPU inference, a 2048-pixel processing limit and 1024-pixel tiles with 32-pixel overlap. The checkpoint SHA256 is recorded in both source reports. Actual NVIDIA CUDA performance was not measured on this machine. [R6, R8]

3 2 Visual Outputs and Intermediate Representations

The panels below use official test images and saved benchmark outputs. Panel order is identical in both figures. Display resizing preserves aspect ratio; no exposure normalization or vivid finish is applied. The normal-light reference is shown for comparison and was not available to the restoration algorithms.



Figure 2 Extreme low light LOL-v1 test image 493 Input versus deep output hybrid output and reference

Image 493 has mean input luminance 0.0249 and is classified as extremely dark. The saved deep output reaches 22.5786 dB PSNR and 0.6992 SSIM; hybrid reaches 23.1574 dB and 0.6932 SSIM. The hybrid has slightly better PSNR for this image, although its SSIM is slightly lower. This example should not be generalized to the test-set average.



Figure 3 Nonuniform illumination LOL-v1 test image 780 Bright windows beside a dark interior

Image 780 has a nonuniform-illumination score of 0.889. Deep-only output clipping is 0.2446%, compared with 0.0033% for hybrid, while deep-only has higher PSNR at 28.9842 versus 25.6671 dB. The scene illustrates the practical tension between lifting the interior and limiting bright-region saturation; the clipping statistic measures all pixels rather than a manually annotated highlight region.

Repository figure sources: outputs/LOL-v1-final-benchmark/00006 and 00013. Their original.png, deep.png and hybrid.png files correspond to manifest test keys 493 and 780; references are under data/LOL-v1/eval15/high. Per-image values come from the same saved report. [R6]

Intermediate Representations

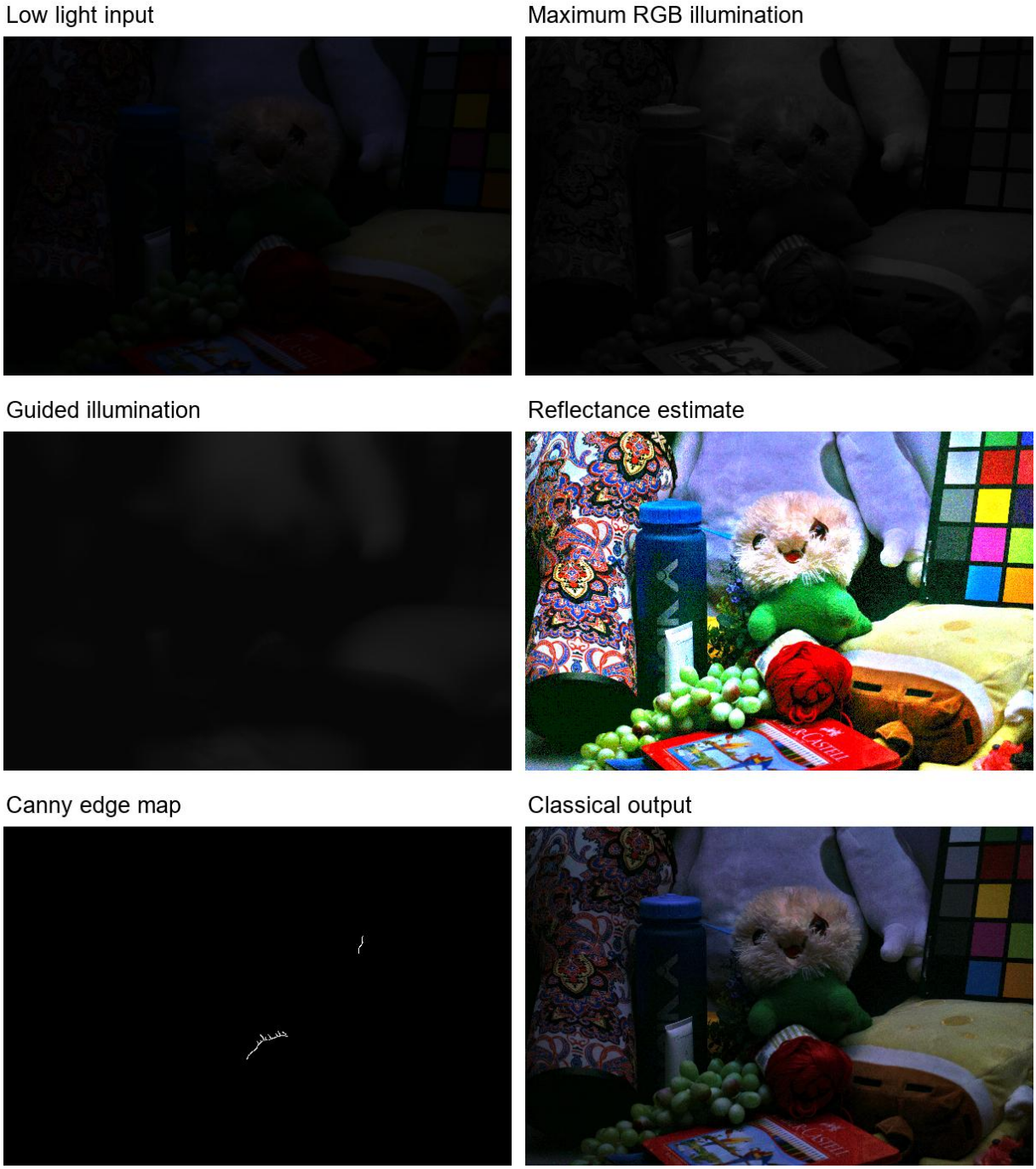


Figure 4 Classical pipeline representations for official test image 493

Maximum-RGB illumination and its guided refinement are grayscale intensities in [0, 1]. Reflectance is the clipped ratio of input RGB to refined illumination. The edge panel is the analyzer's Canny map, not a learned attention map. The final panel is the saved classical output; intermediate maps were regenerated from the same test input using the current classical implementation.

These panels make parameter choices inspectable: illumination separates broad exposure structure, the reflectance estimate exposes noise and surface content, and edges help assess boundary preservation. They are algorithmic estimates, not physical ground truth. No claim is made that the neural model exported these particular maps. [R1, R6]

3 3 Ablation Study

The completed study applies nine variants to the same 15 official test images, producing 135 measured image-variant rows. Classical variants selectively disable denoising, CLAHE, colour correction and safeguards; fusion variants change fixed, global and spatial weighting. Values below are copied from the saved ablation report, with deltas calculated from unrounded means. [R7]

Variant	PSNR dB	SSIM	LPIPS	NIQE	PSNR delta vs full
Retinex only	12.7322	0.5047	0.3461	7.7105	-6.4266
Retinex plus denoising	12.7353	0.5094	0.3416	7.6174	-6.4234
Retinex plus CLAHE	13.5686	0.5111	0.3718	7.9214	-5.5902
Complete classical	13.5286	0.5166	0.3678	7.8276	-5.6302
Deep only	25.1553	0.8441	0.1314	3.4792	+5.9965
Fixed 50 50 fusion	18.4218	0.7412	0.2087	5.6484	-0.7370
Global adaptive fusion	20.4258	0.7908	0.1720	4.6774	+1.2671
Spatial without safeguards	19.1598	0.7847	0.1798	4.3256	+0.0010
Full hybrid	19.1588	0.7847	0.1799	4.3247	+0.0000

Table 2 Nine variant ablation means Higher PSNR and SSIM and lower LPIPS and NIQE are preferred

Measured module effects

Adding bilateral denoising to Retinex alone raises SSIM by 0.0047 and reduces LPIPS by 0.0045, while PSNR changes by only +0.0032 dB. Adding CLAHE to Retinex alone raises PSNR by 0.8364 dB but lowers SSIM by 0.0064 and increases LPIPS by 0.0257. More brightness and contrast therefore do not improve every quality measure.

Spatial fusion without safeguards improves PSNR by 0.7380 dB and SSIM by 0.0435 over fixed 50/50 fusion, while reducing LPIPS by 0.0289. Global adaptive fusion, however, exceeds the full hybrid by 1.2671 dB PSNR and 0.0061 SSIM. Local weighting is therefore not uniformly better on this dataset.

Full safeguards and candidate selection change PSNR by -0.0010 dB relative to spatial fusion without safeguards. Output clipping falls from 0.0109% to 0.0069%, but the aggregate reference metrics are almost unchanged. This comparison changes the safeguard and selection bundle together; it does not isolate the effect of each individual correction.

Removing the classical candidate and controller entirely gives the deep-only result, which exceeds full hybrid by 5.9965 dB PSNR. That comparison removes a complete branch rather than a single learned guidance module. Future tests should separate colour correction, candidate switching and individual safeguards, repeat timings and evaluate additional unseen datasets.

4 Failure Modes and Limitations

Observed hybrid underexposure



Figure 5 Failure example LOL-v1 test image 79 Conservative fusion loses reference fidelity

For image 79, the deep-only candidate scores 27.0771 dB PSNR and 0.9052 SSIM, while hybrid falls to 13.7363 dB and 0.7977. The PSNR gap is 13.3408 dB, the largest deep-versus-hybrid gap in the saved test comparison. The panels show that mixing in the conservative classical result can leave the output darker than the reference. [R6]

Root cause and mitigation

The controller rewards similarity to input edges and colour direction and penalizes clipping and high-frequency residuals. Those checks can mistake useful restoration changes for risk, especially when the input is a poor guide to the desired exposure. A candidate can therefore receive favourable heuristic confidence without matching the normal-light scene well.

A practical current choice is deep mode when reference fidelity is the priority. Further work could calibrate candidate confidence on a separate development dataset, add an exposure-consistency criterion and learn fusion weights with reference supervision. Such changes must be tuned away from the official test set and evaluated on additional datasets before claiming improvement.

Other limitations

- Clipped shadows contain no reliable scene detail. Gain and contrast can reveal existing signal but cannot recover information that was never recorded.
- The Haar residual estimate and artifact maps also respond to texture and exposure changes. They are diagnostic proxies, not calibrated sensor-noise or defect measurements.
- One-epoch adaptation worsened reference metrics and was rejected. The 15-image test set is small, and no cross-dataset robustness claim is supported.
- CUDA execution and browser visual responsiveness remain unverified in the saved evidence. The AMD Radeon 780M in this machine is not a CUDA-capable device.
- The optional 4K export is interpolation to a 3840-pixel long edge, not learned super-resolution. It does not reconstruct absent detail.

5 Deployment and Quickstart Guide

Quickstart commands

Use PowerShell from a fresh checkout. Install Python 3.11, Node.js 22 and uv first. Model downloads require network access during setup; normal inference then uses local weights.

```
# Clone the project
git clone https://github.com/Tanish3115/Robust-Low-Light-Image-Restoration-and-Enhancement.git
cd Robust-Low-Light-Image-Restoration-and-Enhancement

# Create the Python environment and install the backend
uv venv --python 3.11 .venv
uv pip install --python .venv/Scripts/python.exe -r backend/requirements.txt
Copy-Item .env.example .env

# Add neural restoration and perceptual quality metrics
uv pip install --python .venv/Scripts/python.exe `
  -r backend/requirements-deep.txt -r backend/requirements-iqa.txt
.venv/Scripts/python.exe scripts/setup_model.py --download
.venv/Scripts/python.exe scripts/setup_iqa.py --enable

# Build the frontend
cd frontend
npm ci
npm run build
cd ..

# Serve the built application and API
.venv/Scripts/python.exe -m uvicorn backend.app.main:app `
  --host 127.0.0.1 --port 8000
```

Open http://127.0.0.1:8000, upload a low-light image, select a mode and choose Enhance image. API documentation is at http://127.0.0.1:8000/docs. Classical mode works without the optional neural and IQA setup steps.

Deployment format

LumaRestore is a React/Vite frontend with a FastAPI backend and locally executed OpenCV and PyTorch pipelines. The backend serves the production frontend after a build. For development, run npm run dev in a second terminal under frontend and open http://127.0.0.1:5173. No paid AI inference API is required.

A full CPU Docker image and a separate classical build target are provided. Provision the checkpoint and metric assets before starting Compose; it mounts local weights read-only. Stop any native backend already using port 8000, then run:

```
docker compose up --build
```

Reproduce the evaluation

```
.venv/Scripts/python.exe scripts/setup_dataset.py --download
.venv/Scripts/python.exe scripts/benchmark.py `
  --manifest data/LOL-v1/manifest.json --output outputs/report-benchmark `
  --config configs/benchmark.json
.venv/Scripts/python.exe scripts/benchmark.py `
  --manifest data/LOL-v1/manifest.json --output outputs/report-ablations `
  --config configs/benchmark.json --ablations
```

Use new output directories for each run. In the frontend, Performance analytics exposes Run all ablations, optional aligned references, batch experiments and report exports.

Evidence and References

The numerical tables use saved measured reports rather than values copied from a paper. Upstream resources are credited for the architecture and dataset; implementation-specific claims refer to repository modules.

[R1] Implementation modules. backend/src/analysis/analyzer.py; backend/src/classical/pipeline.py; backend/src/deep/inference.py; backend/src/hybrid/controller.py; backend/src/evaluation/experiments.py and metrics.py.

[R2] Retinexformer architecture and pretrained model. Cai et al. Retinexformer One-stage Retinex-based Transformer for Low-light Image Enhancement. ICCV 2023. Official source: https://github.com/caiyuanhao1998/Retinexformer and https://arxiv.org/abs/2303.06705.

[R3] LOL dataset origin. Wei et al. Deep Retinex Decomposition for Low-Light Enhancement. BMVC 2018. Dataset project: https://daooshee.github.io/BMVC2018website/.

[R4] Prepared dataset and split. data/LOL-v1/manifest.json and scripts/setup_dataset.py. Official training pairs 485; local split 437 training and 48 development; official test pairs 15.

[R5] Exploratory training and adaptation. scripts/finetune.py; outputs/LOL-v1-finetuning-validation/config.json and training.csv; outputs/LOL-v1-adaptation-test-comparison; BENCHMARK_RESULTS.md.

[R6] Main benchmark evidence. outputs/LOL-v1-final-benchmark/report.json and per-image output folders. 45 measured rows across three methods. Source run date 6 November 2026.

[R7] Ablation evidence. outputs/LOL-v1-final-ablations/report.json and backend/src/evaluation/experiments.py. 135 measured rows across nine variants.

[R8] Recorded implementation verification. VERIFICATION.md; outputs/device-validation.json; outputs/research-http-verification/verification.json; outputs/docker-research-verification/verification.json.

Recorded verification

The verification record dated 6 November 2026 reports 68 backend tests passed and 17 frontend tests passed, a successful TypeScript/Vite build, native and Docker HTTP checks, genuine neural inference and perceptual metrics, and completed official-test comparisons and ablations. These are recorded checks, not a new full test execution for this report. [R8]

Student Submission Checklist

[x] Repository URL and reproducible quickstart commands included.
[x] Implementation explained without pasting raw source files.
[x] Metric tables traced to measured reports with sample counts and protocol.
[x] Test images and intermediate representations included.
[ ] Enter the official problem ID and exact challenge title.
[ ] Enter registered team name and all member details.
[ ] Add the public demo or video link and confirm repository access.
[ ] Confirm the final submission commit after local frontend text edits.

The evaluated checkpoint fingerprint is 9b4810dc51322fab70be8658ee0ad1df64a54aa814497b7849767baa2ff32cd1. The current base commit is listed on the first page; the saved reports do not record an evaluated Git commit. Large datasets, checkpoints and outputs are ignored by Git, so include the required figures and reports in the submission package or regenerate them using the commands above.